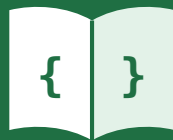

Module 16 — Async, Scheduling and Spring Batch

Exercises

Spring Boot Edu



2026-09-24

Contents

How to Work	2
Exercise 1 — The Cheapest Carrier (Easy)	2
Exercise 2 — An Import that Skips Bad Lines (Medium)	2
Exercise 3 — A Nightly Report Job (Medium)	3
Extra Challenge (Optional)	4

How to Work

1. The starter code is in `modules/16-async-scheduling-batch/exercise/`. Find the TODO comments.
2. The exercises need no Docker: without a database, Spring Batch 6 keeps its job state in memory.
3. Run the tests:

```
./mvnw -Pexercises -pl modules/16-async-scheduling-batch/exercise -am test
```

4. The exercise is done when all tests are green.
5. If you get stuck, read the hints first, then the solution in `modules/16-async-scheduling-batch/solution/`.

Exercise 1 — The Cheapest Carrier (Easy)

Goal: three carriers offer shipping prices. Each answer takes about 300 ms. Ask all of them **in parallel** and take the cheapest offer, even if one carrier fails.

`CarrierClient.quote(carrier, isbn)` is given and already `@Async`.

Tasks (`exercisel.ShippingQuotes.cheapest`):

- TODO 1a — ask every carrier in `CarrierClient.CARRIERS`. Start all calls before waiting for any.
- TODO 1b — a carrier that fails gives no quote. It must not fail the whole search.
- TODO 1c — return the quote with the lowest price.

Hints:

- Lesson section 3.1: first `map(carrier -> carriers.quote(carrier, isbn))` into a list, then `join()` each future.
- `future.exceptionally(error -> null)` turns a failed future into one that completes with null.
- `min(Comparator.comparing(Quote::price))`.

Acceptance criteria: all 3 tests in `ExerciselTest` pass (the parallel one must finish in under 700 ms).

Estimated time: 20 minutes

Exercise 2 — An Import that Skips Bad Lines (Medium)

Goal: an order file contains a few broken lines (no ISBN, quantity “zero”). They must be skipped, and the good lines imported. A file that is mostly broken, however, is probably the wrong file: then the job must fail.

The reader, the processor (which throws `InvalidOrderLineException`) and the writer (`OrderInbox`) are given.

Tasks (exercise2.ImportOrdersJobConfiguration):

- TODO 2a — make the step fault tolerant: skip lines with an `InvalidOrderLineException` ...
- TODO 2b — ... but at most 3 of them. After that, the job fails.

Hints:

- Lesson section 3.4: `.faultTolerant().skip(...).skipLimit(...)` after the writer.
- `orders.csv` has 2 bad lines (→ COMPLETED, 5 orders), `orders-mostly-broken.csv` has 4 (→ FAILED).
- There are two jobs in this project, so the tests choose one with `jobs.setJob(...)`.

Acceptance criteria: both tests in `Exercise2Test` pass.

Estimated time: 20 minutes

Exercise 3 — A Nightly Report Job (Medium)

Goal: every night at 02:00, a report of the previous day's sales is created. The work is a batch job with the job parameter `report.date`, so any day can be reported again by hand. The scheduler only starts the job.

Tasks (package exercise3):

- TODO 3a — `DailySalesReportJobConfiguration`: the tasklet counts the sales of date, adds up their amounts and stores a `DailyReport` in the archive.
- TODO 3b — `NightlyReportScheduler.createReport`: run every night at 02:00 Istanbul time.
- TODO 3c — start `dailySalesReportJob` with the job parameter `report.date = yesterday`, according to the clock.

Hints:

- The tasklet already receives the date: `@Value("#{jobParameters['report.date']}") LocalDate date`.
- `new JobParametersBuilder().addLocalDate("report.date", date).toJobParameters()`, then `jobs.start(job, parameters)`.
- Use the injected `Clock`: `LocalDate.now(clock).minusDays(1)`. The test sets the clock to 02:00 on 25 September, so the report must be for 24 September.
- Lesson section 3.2: `@Scheduled(cron = "0 0 2 * * *", zone = "Europe/Istanbul")`.

Acceptance criteria: both tests in `Exercise3Test` pass.

Estimated time: 30 minutes

Extra Challenge (Optional)

Start the lesson application twice (two terminals, `--server.port=8081` for the second one). Change `bookstore.report.cron` so that the report runs every minute, and watch it run in **both** instances. Then protect it with ShedLock (`shedlock-spring` and `shedlock-provider-jdbc-template`, with the lock table in PostgreSQL) and check that it runs only once per minute.